

Multi Agent RAG based HR Support Assistant

Final Capstone Project

by **Rajat Tyagi**

Employee ID: 2261727

A 3-agent Retrieval-Augmented Generation (RAG) chatbot that answers HR policy questions from company PDF documents, guarded by input/output safety checks, and automatically escalates unresolved queries to the right team (IT, Payroll, or HR) via email.

Table of Contents

- 1. Project Overview
- 2. Objectives
- 3. System Architecture
- 4. Multi-Agent Design
- 5. Guardrails & Safety
- 6. Escalation & Email Workflow
- 7. Technology Stack
- 8. Project Structure
- 9. Configuration (Environment Variables)
- 10. Setup & Installation
- 11. Running the Application
- 12. Ingesting HR Policy Documents
- 13. Testing
- 14. API Reference
- 15. Learning Outcomes & Future Enhancements
- 16. Conclusion

1. Project Overview

The HR Support GenAI Assistant is a learning-oriented capstone project that demonstrates how to build a production-style Retrieval-Augmented Generation (RAG) chatbot for internal HR support. Employees can ask natural-language questions about leave, payroll, benefits, onboarding, and company policy. The system retrieves the most relevant passages from ingested HR policy PDFs, grounds the LLM's answer strictly in that content, and — when it cannot find a confident, grounded answer — automatically drafts and sends a professional escalation email to the correct internal team (IT, Payroll, or HR) after confirming with the employee.

The project intentionally keeps its dependency footprint small (regex-based guardrails instead of a heavyweight moderation library, MailHog instead of a production mail server) so every moving part remains easy to read, run locally, and reason about end-to-end.

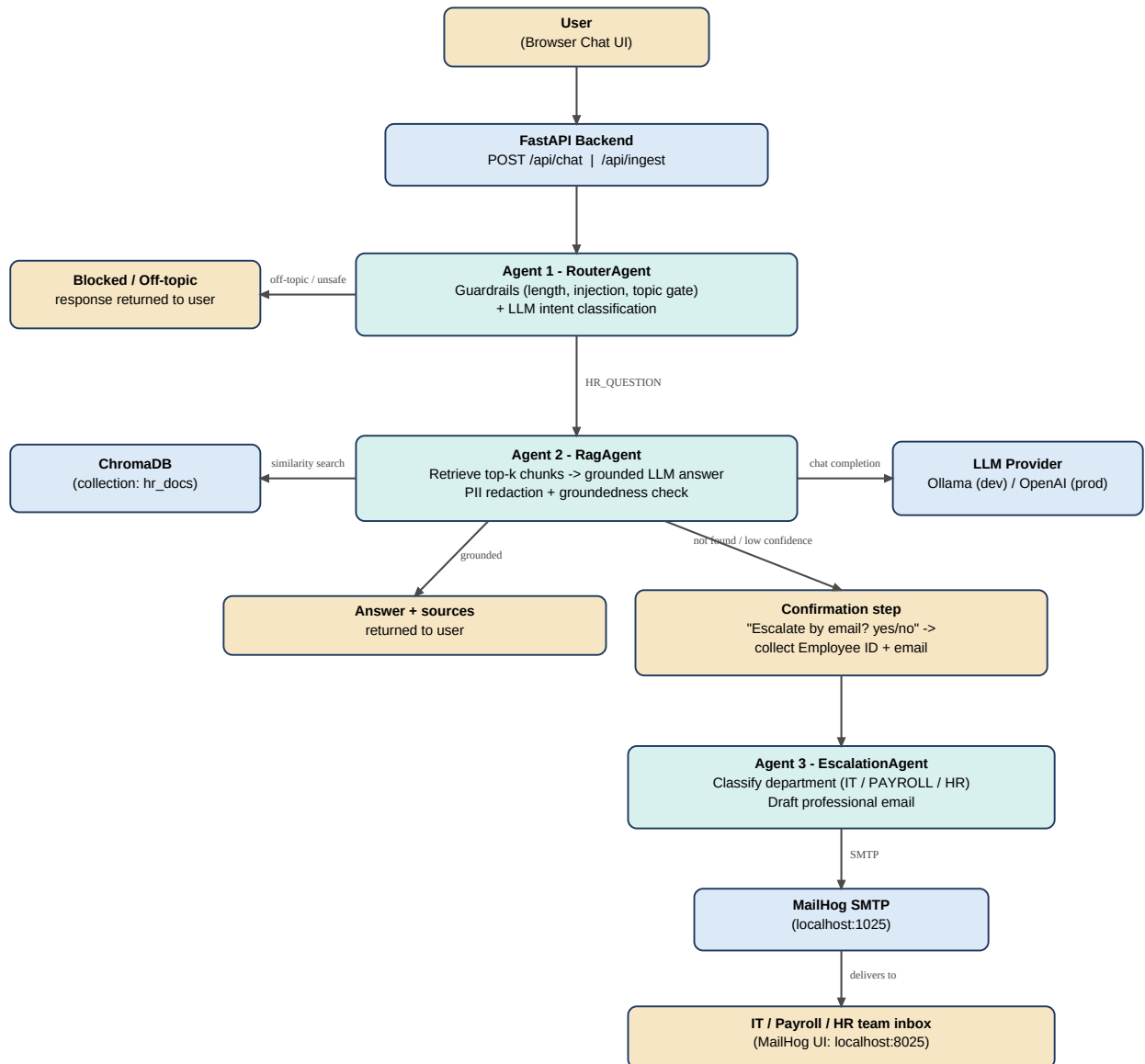
2. Objectives

- Build a multi-agent architecture where each agent has a single, well-defined responsibility.
- Ground every answer in verifiable source documents to avoid hallucinated HR policy advice.
- Apply layered guardrails: input validation, prompt-injection detection, topic gating, PII redaction, and output groundedness checks.
- Support a transparent dev/prod switch between a free local LLM (Ollama) and a hosted LLM (OpenAI) without changing application code.
- Automate escalation of unanswered queries to the correct business team via a human-in-the-loop (confirm before sending) email workflow.
- Ship a minimal, dependency-light web UI for chatting and uploading policy PDFs.

3. System Architecture

The request flow is a straight pipeline through three cooperating agents sitting behind a single FastAPI endpoint. Agent 1 gates and classifies the request, Agent 2 performs retrieval-grounded answering, and Agent 3 handles escalation and email notification when Agent 2 cannot help.

System Architecture



4. Multi-Agent Design

4.1 Agent 1 — RouterAgent

File: *app/agents/router_agent.py*. This is the first line of defense. It runs input guardrails (message length, prompt-injection pattern matching) and a cheap keyword pre-filter for obvious HR topics before falling back to an LLM call that classifies the message into one of four intents.

Intent	Meaning	Behaviour
HR_QUESTION	A genuine HR/policy/payroll/benefits question	Forwarded to Agent 2 (RagAgent)
GREETING	Small talk / hello	Answered directly with a friendly greeting
OFF_TOPIC	Unrelated to HR (coding help, trivia, etc.)	Blocked with a polite refusal
UNSAFE	Jailbreak attempts, abuse, secret extraction	Blocked before reaching any LLM/RAG call

4.2 Agent 2 — RagAgent

File: *app/agents/rag_agent.py*. Performs the core RAG loop:

- Runs a similarity search against ChromaDB (collection *hr_docs*) and keeps only chunks whose relevance score passes *MIN_RELEVANT_SCORE*.
- If no relevant chunk is found, it does not answer from memory — it asks the user for confirmation before escalating.
- Otherwise, it prompts the LLM with a strict system instruction to answer only from the retrieved context and cite the source file name(s).
- Redacts PII (SSN, email, phone, credit card) from the generated answer.
- Runs a groundedness check; low-confidence/ungrounded answers also trigger the escalation confirmation flow instead of being shown to the user as fact.

4.3 Agent 3 — EscalationAgent

File: *app/agents/escalation_agent.py*. Once the employee confirms they want to escalate and supplies their Employee ID and official email address, this agent:

- Classifies the request into exactly one owning team — IT, PAYROLL, or HR — using a dedicated LLM prompt.
- Looks up that team's mailbox from configuration (IT_TEAM_EMAIL / PAYROLL_TEAM_EMAIL / HR_TEAM_EMAIL).
- Drafts a complete, professional email with a descriptive subject line ("HR Support Escalation - <Employee ID> needs <Team> assistance"), the employee's ID/email, the escalation reason, the original (PII-redacted) request, and a Reply-To header set to the employee's email so the team can respond directly.
- Sends the email over SMTP — MailHog for local development, any real SMTP server in production.

5. Guardrails & Safety

File: `app/guardrails.py`. Implemented with plain regular expressions on purpose, to keep the safety layer transparent and dependency-free for a learning project.

- **Length check** — rejects messages longer than `MAX_INPUT_CHARS`.
- **Prompt-injection detection** — blocks phrases like "ignore previous instructions", "developer mode", or requests to reveal the system prompt.
- **Topic gate** — a keyword allow-list (leave, payroll, benefits, policy, etc.) used as a fast pre-filter before the LLM intent classification.
- **PII redaction** — regex-based redaction of SSNs, emails, phone numbers, and credit-card-like numbers from both escalation emails and generated answers.
- **Groundedness check** — rejects answers that appear confident but were not backed by retrieved context.

6. Escalation & Email Workflow

Escalation is a human-in-the-loop, multi-turn conversation rather than a silent background action, so the employee always stays in control of whether an email is sent on their behalf. State is kept in-memory per `session_id` in `app/main.py`.

- **Step 1 — Confirm:** RagAgent could not answer confidently, so the bot asks: "...Would you like me to notify the relevant team by email? (yes/no)".
- **Step 2 — Collect details:** if the user says yes, the bot asks for their Employee ID and official email address, e.g. "`EMP1234, jane.doe@company.com`".
- **Step 3 — Send:** once both values are parsed successfully, EscalationAgent classifies the department and sends the drafted email; the bot confirms back to the user which team was notified.
- If the user declines at Step 1, or the details cannot be parsed, no email is ever sent.

7. Technology Stack

Layer	Technology
API framework	FastAPI + Uvicorn
Orchestration	LangChain (langchain, langchain-community, langchain-text-splitters)
LLM (dev)	Ollama — llama3.1 (chat), nomic-embed-text (embeddings), local & free
LLM (prod)	OpenAI — gpt-4o-mini (chat), text-embedding-3-small (embeddings)
Vector store	ChromaDB, persisted to <code>data/chroma_db</code> , collection <code>hr_docs</code>
PDF ingestion	pypdf + LangChain text splitters
Validation	Pydantic v2 / pydantic-settings
Guardrails	Custom regex-based checks (no extra dependency)
Email delivery (dev)	MailHog (SMTP catch-all on 1025, web UI on 8025)

Layer	Technology
Frontend	Vanilla HTML / CSS / JavaScript
Testing	pytest, httpx

8. Project Structure

```
app/  
agents/  
router_agent.py # Agent 1: guardrails + intent routing  
rag_agent.py # Agent 2: retrieval + grounded answer + escalation trigger  
escalation_agent.py # Agent 3: department classification + email notification  
config.py # env-driven settings (dev/prod provider switch, SMTP)  
llm.py / embeddings.py # provider factories (Ollama <-> OpenAI)  
vectorstore.py # Chroma wrapper  
ingestion.py # PDF -> chunks -> embeddings -> Chroma  
guardrails.py # PII redaction, injection/topic checks  
schemas.py # pydantic request/response models  
main.py # FastAPI app (chat, ingest, health endpoints)  
static/ # minimal HTML/CSS/JS chat UI  
data/  
pdfs/ # drop HR policy PDFs here (or upload via UI)  
chroma_db/ # persisted vector store (gitignored)  
scripts/  
ingest.py # CLI: ingest every PDF in data/pdfs  
tests/  
test_guardrails.py
```

9. Configuration (Environment Variables)

All settings live in `app/config.py` and are loaded from a local `.env` file via `pydantic-settings`.

Variable	Purpose	Default
ENV	"dev" -> Ollama, "prod" -> OpenAI	dev
LLM_PROVIDER	Force "ollama" or "openai" explicitly	(unset)
OLLAMA_BASE_URL / OLLAMA_MODEL / OLLAMA_EMBED_MODEL	Local model config	llama3.1 / nomic-embed-text
OPENAI_API_KEY / OPENAI_MODEL / OPENAI_EMBED_MODEL	Hosted model config	gpt-4o-mini / text-em bedding-3-small
CHROMA_DIR / PDF_DIR	Storage locations	data/chroma_db / data/pdfs
TOP_K / MAX_INPUT_CHARS / MIN_RELEVANT_SCORE	RAG & guardrail tuning	4 / 2000 / 0.3
SMTP_HOST / SMTP_PORT / SMTP_FROM	Outbound mail server for escalations	localhost / 1025
IT_TEAM_EMAIL / PAYROLL_TEAM_EMAIL / HR_TEAM_EMAIL	Escalation recipients per department	example.com addresses

10. Setup & Installation

10.1 Create and activate a virtual environment, then install dependencies:

```
python -m venv .env  
.env\Scripts\Activate.ps1  
pip install -r requirements.txt
```

10.2 Copy the environment template:

```
copy .env.example .env
```

10.3 Dev mode (local, free) — install Ollama and pull the models used:

```
ollama pull llama3.1  
ollama pull nomic-embed-text
```

Keep ENV=dev in .env (default).

10.4 Prod mode (OpenAI) — set in .env:

```
ENV=prod  
OPENAI_API_KEY=sk-...
```

10.5 Escalation email (local dev) — run MailHog to catch escalation emails:

```
docker run -d --name mailhog -p 1025:1025 -p 8025:8025 mailhog/mailhog
```

Then open <http://localhost:8025> to view every escalation email sent by the assistant.

SMTP_HOST/SMTP_PORT in .env already default to MailHog's localhost:1025.

11. Running the Application

```
uvicorn app.main:app --reload
```

Then open <http://127.0.0.1:8000> in a browser and start chatting.

12. Ingesting HR Policy Documents

Option A — CLI: drop PDFs into data/pdfs/ then run:

```
python scripts/ingest.py
```

Option B — Web UI: drag-and-drop PDFs from the running chat UI (calls POST `/api/ingest`).

13. Testing

```
pytest
```

tests/test_guardrails.py covers the regex-based guardrail logic (length limits, prompt-injection detection, PII redaction, HR keyword gating).

14. API Reference

Endpoint	Method	Description
/	GET	Serves the chat web UI (index.html)
/api/health	GET	Returns service status and active LLM provider
/api/chat	POST	Main chat endpoint; drives routing, RAG answering, and the escalation confirm/collect/send state machine
/api/ingest	POST	Uploads one or more PDFs, chunks and embeds them into ChromaDB

15. Learning Outcomes & Future Enhancements

What this project demonstrates:

- Designing a multi-agent pipeline where each agent has a single responsibility.
- Grounding LLM output in retrieved evidence and refusing to answer when evidence is insufficient.
- Layered, dependency-light guardrails suitable for learning before adopting a full framework.
- A configurable, provider-agnostic LLM/embedding layer (local vs. hosted).
- A human-in-the-loop escalation workflow with professional, automatically drafted email.

Potential next steps:

- Swap the keyword-based topic gate for a proper classifier or a library such as guardrails-ai or nemo-guardrails.
- Persist conversation memory beyond a single stateless message per turn.
- Add authentication/authorization before exposing this beyond local experimentation.
- Model the router -> RAG -> escalation flow explicitly as a graph (e.g. LangGraph).
- Replace MailHog with a production SMTP provider and add retry / dead-lettering for failed escalation emails.

16. Conclusion

This capstone project delivers a working, end-to-end example of a safe, grounded, multi-agent RAG assistant for internal HR support — covering retrieval-augmented answering, layered guardrails, and an automated yet consent-driven escalation workflow to the right internal team by email. It is intentionally small and readable so every design decision can be traced back to a single file.